

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1

За седьмой семестр

Тема: «Реализация многослойного персептрона для решения задачи
«исключающее ИЛИ»

Выполнил:
Студент 4 курса
Группы ИИ-26
Ковальчук А.И.
Проверил:
Никитин С.К.

Цель: Реализовать многослойный персептрон (MLP) архитектуры 2-2-1 с сигмоидной функцией активации для решения задачи типа «исключающее ИЛИ» (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10, соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с однослойным персептроном, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона [-10; 10] (не обязательно совпадающих с s_0 и s_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (s_0 или s_1) оно ближе.

7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Вариант 7

A	B	$A \oplus B$
-4	-4	-4
-4	-9	-9
-9	-4	-9
-9	-9	-4

Архитектура

Многослойный персептрон (МСП). Архитектура 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон). На всех слоях используется сигмоидная функция активации. Обучение проводится методом обратного распространения ошибки (онлайн-режим). Функция потерь — MSE. Для предотвращения затухания градиента входные данные нормализованы (деление на 10), а целевые значения переведены в классы 0 и

Однослойный персептрон (ОСП). Архитектура 2-1 (2 входных нейрона, 1 выходной). Функция активации — сигмоида.

Результаты обучения

MLP:

Сходимость достигнута на 1316 эпохе.

Итоговая ошибка 0.009988.

Accuracy 100%.

SLP:

Сходимость не достигнута, т. к. ошибка остаётся высокой.

Итоговая ошибка 0.500002.

Accuracy 75%.

Примеры работы обученной сети:

Вход (А,В) Выход сети $\hat{y}$ Ближе к классу

(-4, -9)	-8.2623	-9
(-5, -5)	-4.3811	-4
(-6, -7)	-4.4454	-4
(-7, -6)	-4.5222	-4

Код программы

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

class DenseLayer:
    def __init__(self, units=1, activation='relu'):
        self.units = units
        self.activation = activation.lower()
        self.w = None
        self.b = None
        self.input = None
        self.z = None

    def init_weights(self, fan_in):
        if self.activation in ['relu', 'leaky_relu']:
            std = np.sqrt(2.0 / fan_in)
        else:
```

```
std = np.sqrt(1.0 / fan_in)
```

```
self.w = np.random.normal(0.0, std, (fan_in, self.units))
```

```
self.b = np.zeros(self.units)
```

```
def forward(self, x):
```

```
    if self.w is None:
```

```
        self.init_weights(x.shape[-1])
```

```
    self.z = x @ self.w + self.b
```

```
    if self.activation == 'relu':
```

```
        return np.maximum(0, self.z)
```

```
    elif self.activation == 'leaky_relu':
```

```
        return np.maximum(0.01 * self.z, self.z)
```

```
    elif self.activation == 'sigmoid':
```

```
        return 1 / (1 + np.exp(-np.clip(self.z, -500, 500)))
```

```
    elif self.activation == 'tanh':
```

```
        return np.tanh(self.z)
```

```
    elif self.activation == 'softmax':
```

```
        exp_z = np.exp(self.z - np.max(self.z, axis=1, keepdims=True))
```

```
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

```
elif self.activation == 'linear':
```

```
    return self.z
```

```
else:
```

```
    raise ValueError(f"Неизвестная активация: {self.activation}")
```

```
def derivative(self, a):
```

```
    if self.activation == 'relu':
```

```
        return (self.z > 0).astype(float)
```

```
    elif self.activation == 'leaky_relu':
```

```
        return (self.z > 0).astype(float) + 0.01 * (self.z <= 0).astype(float)
```

```
    elif self.activation == 'sigmoid':
```

```
        return a * (1 - a)
```

```
    elif self.activation == 'tanh':
```

```
        return 1 - a**2
```

```
    elif self.activation == 'linear':
```

```
        return np.ones_like(a)
```

```
    elif self.activation == 'softmax':
```

```
        return np.ones_like(a)
```

```
    return np.ones_like(a)
```

```
class Input:
    def __init__(self, shape=None):
        self.shape = shape

    def forward(self, x):
        if self.shape is not None:
            expected = self.shape if isinstance(self.shape, tuple) else (self.shape,)
            if x.shape[1:] != expected:
                x = x.reshape((x.shape[0],) + expected)
        return x
```

```
class Dropout:
    def __init__(self, rate=0.5):
        self.rate = rate
        self.mask = None
        self.training = True

    def forward(self, x):
        if not self.training:
            return x
        self.mask = (np.random.rand(*x.shape) > self.rate) / (1 - self.rate)
        return x * self.mask
```

```
class Sequential:
```

```

def __init__(self, layers):
    self.layers = layers
    self.history = []
    self.loss_fn = None
    self.loss_deriv = None
    self.l1 = 0
    self.l2 = 0

def compile(self, loss='mse', l1=0.0, l2=0.0):
    self.l1 = l1
    self.l2 = l2
    self.loss = loss.lower()

    if self.loss == 'mse':
        self.loss_fn = lambda y, p: 0.5 * np.sum((p - y) ** 2)
        self.loss_deriv = lambda y, p: (p - y)

    elif self.loss == 'mae':
        self.loss_fn = lambda y, p: np.mean(np.abs(p - y))
        self.loss_deriv = lambda y, p: np.sign(p - y)

    elif self.loss == 'binary_crossentropy':
        eps = 1e-8
        self.loss_fn = lambda y, p: -np.mean(
            y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps)
        )
        self.loss_deriv = lambda y, p: (p - y)

```

```
elif self.loss == 'categorical_crossentropy':
```

```
    eps = 1e-8
```

```
    self.loss_fn = lambda y, p: -np.mean(np.sum(y * np.log(p + eps), axis=1))
```

```
    self.loss_deriv = lambda y, p: (p - y)
```

```
else:
```

```
    raise ValueError(f"Неизвестная функция потерь: {loss}")
```

```
def forward(self, x):
```

```
    out = x
```

```
    for layer in self.layers:
```

```
        out = layer.forward(out)
```

```
    return out
```

```
def fit(self, x_input, y_input, epochs=100, alpha=0.001, batch_size=32,
```

```
    clip_value=5.0, adaptive_alpha=False, Ee=1e-3):
```

```
    x_input = np.asarray(x_input, dtype=np.float32)
```

```
    y_input = np.asarray(y_input, dtype=np.float32).reshape(-1, 1)
```

```
    n_samples = x_input.shape[0]
```

```
    self.history = []
```

```
    for epoch in range(epochs):
```

```
        indices = np.random.permutation(n_samples)
```

```
        x_shuffled = x_input[indices]
```

```
        y_shuffled = y_input[indices]
```

```

for i in range(0, n_samples, batch_size):
    x = x_shuffled[i:i + batch_size]
    y = y_shuffled[i:i + batch_size]

    if adaptive_alpha:
        alpha_t = 1.0 / (1 + np.sum(x ** 2))
    else:
        alpha_t = alpha

    activations = [x]
    for layer in self.layers:
        if isinstance(layer, Dropout):
            layer.training = True
        activations.append(layer.forward(activations[-1]))

    pred = activations[-1]
    delta = self.loss_deriv(y, pred)

    for l in range(len(self.layers) - 1, -1, -1):
        layer = self.layers[l]

        if isinstance(layer, Input):
            continue

        if isinstance(layer, Dropout):
            delta *= layer.mask

```

```
continue
```

```
a_prev = activations[l]
```

```
da = layer.derivative(activations[l + 1])
```

```
delta *= da
```

```
grad_w = (a_prev.T @ delta) / x.shape[0]
```

```
grad_b = np.mean(delta, axis=0)
```

```
if self.l2:
```

```
    grad_w += 2 * self.l2 * layer.w
```

```
if self.l1:
```

```
    grad_w += self.l1 * np.sign(layer.w)
```

```
grad_w = np.clip(grad_w, -clip_value, clip_value)
```

```
grad_b = np.clip(grad_b, -clip_value, clip_value)
```

```
previous_delta = delta @ layer.w.T
```

```
layer.w -= alpha_t * grad_w
```

```
layer.b -= alpha_t * grad_b
```

```
delta = previous_delta
```

```
epoch_loss = self.loss_fn(y_input, self.predict(x_input))
```

```
self.history.append(epoch_loss)
```

```
if epoch % 10 == 0 or epoch == epochs - 1:
```

```
    print(f"Epoch {epoch:4d} | Es = {epoch_loss:.6f}")
```

```
if epoch_loss <= Ee:
    print(f"Достигнут критерий остановки: {epoch_loss} <= {Ee}")
    return epoch + 1, True
```

```
return epochs, False
```

```
def predict(self, X):
```

```
    X = np.asarray(X, dtype=np.float32)
```

```
    if X.ndim == 1:
```

```
        X = X.reshape(1, -1)
```

```
    for layer in self.layers:
```

```
        if isinstance(layer, Dropout):
```

```
            layer.training = False
```

```
    return self.forward(X)
```

```
def evaluate(self, X, Y):
```

```
    Y = np.asarray(Y, dtype=np.float32).reshape(-1, 1)
```

```
    preds = self.predict(X)
```

```
    loss = self.loss_fn(Y, preds) if self.loss_fn is not None else None
```

```
    last_activation = getattr(self.layers[-1], "activation", None)
```

```
    if last_activation == "sigmoid":
```

```
        y_hat = (preds >= 0.5).astype(np.float32)
```

```
acc = np.mean(y_hat == Y)

else:

    acc = None

return loss, acc
```

Вывод:

- 1) **Смог ли ОСП решить задачу?** Нет. ОСП способен строить только линейные разделяющие границы (прямые линии). Данные задачи XOR линейно неразделимы, поэтому ОСП не может отделить классы друг от друга.
- 2) **Смог ли МСП решить задачу?** Да, МСП успешно справился с задачей за 1316 эпох.
- 3) **Влияние небинарного диапазона и выбор активации.** Так как по условию требовалось использовать сигмоиду (выдающую значения строго от нуля до единицы), произвольные числа потребовали обязательной нормализации. Без масштабирования входов и маппинга выходов в логические 0 и 1 обучение было невозможно из-за паралича сети (затухания градиентов).
- 4) **Зачем нужен скрытый слой?** Скрытый слой трансформирует исходное линейно неразделимое пространство признаков в новое пространство, в котором классы можно разделить простой гиперплоскостью.
- 5) **Сложность по сравнению с классическим XOR.** Логическая сложность задачи осталась прежней (распределение точек не изменилось), однако вычислительная сложность возросла: большие входные значения потребовали предварительной обработки данных для корректной работы математического аппарата сети.